# robots_moviles_25_26

# Práctica 2: Mapeado, localización y navegación con los algoritmos integrados en ROS2

> Esta práctica se debe realizar y entregar de forma **individual** (salvo el apartado de pruebas con los robots reales que se hará en grupo).

En esta práctica probaremos los algoritmos de mapeado, localización y navegación que ya vienen integrados en ROS2.

Daremos aquí los pasos a seguir con el simulador **mvsim**. Si prefieres usar otro simulador, en Internet hay muchos tutoriales que generalmente usan Gazebo. Puedes seguirlos en lugar de estos pasos. Solo tienes que asegurarte que sean tutoriales de **Nav2** (el sistema de ROS2, no valen tutoriales de ROS1) y que expliquen 3 cosas: cómo construir y guardar un mapa del entorno, cómo localizarse y cómo navegar (fijar un destino en el mapa y que el robot vaya automáticamente).

## Paso previo: instalación de los paquetes necesarios

En las siguientes instrucciones, sustituye `<ros2-distro>` por la distribución de ROS2 que tengas: jazzy, humble, …

Para instalar `slam_toolbox`, la herramienta para crear mapas del entorno:

```
sudo apt install ros-<ros2-distro>-slam-toolbox
```

Para instalar nav2, el sistema de navegación de ROS2 (incluye algoritmos de localización, planificación y seguimiento de trayectorias, evitación de obstáculos,…)

```
sudo apt install ros-<ros2-distro>-navigation2
sudo apt install ros-<ros2-distro>-nav2-bringup
```

## Mapeado (hasta 2 puntos)

Lo primero que necesitamos para localizarnos y navegar es un mapa. Usaremos alguno de los algoritmos de construcción de mapas ya integrados en ROS2. Actualmente, el paquete más

usado para mapear en ROS2 es `slam_toolbox`. Este paquete genera mapas de tipo “rejilla de ocupación”, que son los más comunes en robótica móvil.

> Como su nombre indica, `slam_toolbox` implementa SLAM (Simultaneous Localization And Mapping), es decir, mientras va construyendo el mapa también se va localizando, aunque en esta primera sección solo nos interesa la parte de mapeado. Una vez construido todo el mapa se puede usar solo en modo localización o en modo “Lifelong mapping” en el que continuamente está refinando el mapa por si hay cambios.

Un mundo fácil de mapear es el `demo_turtlebot_world` incluido en las demos de mvsim, puedes lanzarlo con:

```
ros2 launch mvsim demo_turtlebot_world.launch.py do_fake_localization:=false use_r
```

El parámetro `do_fake_localization:=false` hace que el simulador no calcule la localización del robot. No queremos que lo haga ya que lo haremos nosotros. por otro lado con `use_rviz:=false` decimos que no arranque rviz2, es mejor arrancarlo nosotros manualmente ya que así podemos mostrar solo las visualizaciones que necesitemos.

A lo largo de toda la práctica veremos que los algoritmos de mapeado, localización y navegación usan unos ficheros de configuración con formato .yaml. Puedes bajártelos desde moodle en un .zip o desde este enunciado.

En concreto el mapeado usa un archivo de configuración `slam.yaml`. El parámetro más importante es `scan_topic` que debe coincidir con el topic del laser 2D. En el caso del `demo_turtlebot_world` es `/laser1`, pero si usas otro mundo deberías cambiarlo por el topic correspondiente.

```yaml
slam_toolbox:
  ros__parameters:
    use_sim_time: false     # Si el simulador publica /clock, aquí se pone true (p
    odom_frame: odom
    base_frame: base_link
    map_frame: map
    scan_topic: /laser1     # El laser2d del mundo "demo_turtlebot_world.world.xml
    mode: mapping
```

Para lanzar el algoritmo de mapeado ejecuta en otra terminal

```
ros2 launch slam_toolbox online_async_launch.py slam_params_file:=./slam.yaml
```

suponiendo que el archivo `slam.yaml` lo tienes en el directorio actual y de ahí el `./` delante del nombre. Si está en otro directorio tendrás que poner la trayectoria para llegar hasta él.

En otra terminal puedes abrir rviz2 tecleando `rviz2`. Añade un *display* para ver el mapa (botón `Add` en la ventana `displays`, solapa `by topic` y elige el tipo `Map` dentro del topic `/map`). Este topic lo publica el algoritmo de mapeado. Verás el trozo de mapa que se puede construir desde la posición inicial del robot. Ve moviendo al robot con el teclado para ir ampliando el mapa. Ten en cuenta que a veces el mapa tarda unos pocos segundos en actualizarse.

Cuando veas que ya se ha mapeado todo el entorno puedes guardar el mapa ejecutando en otra terminal (no pares el proceso de mapeado):

```
ros2 run nav2_map_server map_saver_cli -f mimapa  #o el nombre que le quieras dar
```

Verás que en el directorio actual aparece un par de archivos nuevos, con extensión `.yaml` y `.pgm`. El `yaml` es un archivo con metadatos del mapa y el `pgm` es el mapa en forma de imagen, cada pixel se corresponde con una celdilla de la rejilla de ocupación.

**Entregable** busca información sobre qué significa cada uno de los parámetros que aparece en el .yaml del mapa y cuéntalo en la documentación poniendo como ejemplo tu propio yaml.

**Entregable** guarda mapas de al menos el `turtlebot_world` y el mundo de la práctica 1 (CUIDADO, ten en cuenta que aquí el topic del laser es `/scan_raw` y tendrás que cambiarlo en el `slam.yaml`). Explica en la documentación si el mapa sale correctamente o hay algún tipo de fallo y en ese caso por qué crees que podría fallar el algoritmo de mapeado.

**Entregable** busca información sobre cómo funciona el algoritmo de mapeado de `slam_toolbox` y pon una explicación de al menos un párrafo en la documentación. No es necesario poner fórmulas sino dar una explicación intuitiva de cómo se construye el mapa (¿usa partículas, un filtro de bayes, algo que hayamos visto en clase o es totalmente diferente?).

## Localización (hasta 2 puntos)

Una vez tenemos un mapa del entorno podemos usarlo para mantener localizado al robot en todo momento sin depender de la odometría.

> En el simulador es difícil ver las ventajas de los algoritmos de localización sobre la odometría, ya que ambos tendrán la misma precisión, pero en el mundo real el error de la odometría siempre crece con el movimiento mientras que la localización va a mantener el error dentro de ciertos márgenes siempre que tenga puntos de referencia (que no esté en una zona en la que el sensor no detecte nada).

Para que esto funcione, necesitas haber construido el mapa en el paso anterior y tenerlo guardado en un archivo `.yaml`. También necesitarás un archivo de configuración `.yaml` para el algoritmo de localización:

```
#archivo amcl.yaml mínimo
amcl:
  ros__parameters:
    scan_topic: /laser1
```

ROS lleva integrado un algoritmo de localización basado en un filtro de partículas llamado AMCL (Adaptive Monte Carlo Localization). Para ponerlo en marcha escribe en una terminal:

```
ros2 launch nav2_bringup localization_launch.py map:=./mimapa.yaml params_file:=./a
```

Sustituye el `./` por la trayectoria a seguir hasta donde estén los archivos si no es el directorio actual.

En otra terminal abre rviz2 con el comando `rviz2`. Verás que aparece un error en rojo “Global Status: Error” y que pone algo como “Frame [map] does not exist”. Esto es porque el algoritmo de localización todavía no está inicializado y ROS no sabe en qué posición está el robot con respecto al mapa. Tenemos que indicar nosotros cuál es esa posición inicial.

En rviz añade un display `by topic` con el tipo `Map` dentro del topic `/map`. Una vez añadido, en el panel de display, dentro del nuevo `Map`, en el apartado `Topic` asegúrate de cambiar el `Durability Policy` a `Transient Local`, si no el mapa no aparecerá.

> En ROS existen distintas “políticas de durabilidad”: qué pasa si se publica un mensaje cuando todavía no ha llegado ningún suscriptor. `Volatile` significa que si un suscriptor empieza a escuchar “tarde” ya no recibirá los mensajes anteriores mientras que `Transient Local` indica que se guardan y los recibirá el nuevo suscriptor. La cuestión es que **si el “publicador” publica los mensajes en un modo (en el caso del mapa `Transient Local`), el suscriptor (aquí RViz) tiene que usar el mismo modo para recibirlos**. Y también pasa lo mismo con otros parámetros del mensaje como la “reliability”. Así que veréis que muchas veces nos tocará cambiar la opción en RViz al tipo que use el *publisher*.

Ahora hay que marcar la posición inicial del robot, con la opción de la barra de herramientas `2D pose estimate`. Pulsa sobre ella y luego en el mapa, clica en la posición que creas que está el robot y arrastra hacia la dirección en la que mira, verás que aparece una flecha verde indicando esta posición y dirección. Si te equivocas hazlo otra vez. Al hacerlo el AMCL empieza a emitir la transformación map->odom y el error de fixed frame en rviz debería dejar de aparecer.

Para ver la posición estimada en todo momento por el algoritmo AMCL, añadir un display `by topic -> /amcl_pose / PoseWithCovariance`. Cambiar el `Durability Policy` a `Transient Local` igual que hiciste con el mapa. Se verá la posición estimada con una flecha y una elipse de incertidumbre inicialmente con un error grande. Si vas moviendo al robot con el teclado desde el simulador la incertidumbre debería decrecer conforme el robot se vaya localizando (aunque siempre va a haber un cierto error).

AMCL es un algoritmo de localización basado en un filtro de partículas como el que vimos en clase de teoría. Para ver la nube de partículas: añadir display (en este caso tiene que ser `by display type` ya que el topic no aparece por defecto). Dentro de `nav2_rviz_plugins` seleccionar `ParticleCloud`. El topic debe ser `/particle_cloud` y en Reliability: BEST_EFFORT y Durability: VOLATILE. Cuando añadas el display ten en cuenta que inicialmente no verás las partículas si el robot no se está moviendo porque amcl no las publica cuando el robot está quieto. Luego ya se verán todo el rato. Si haces zoom verás que por defecto son como pequeñas flechas indicando posibles posiciones y orientaciones del robot.

**Entregable:** Incluye un pequeño video de unos 5 segundos de duración donde se vea el robot moviéndose en el simulador y a la vez la ventana de rviz con la posición calculada por el algoritmo de localización y la nube de partículas.

**Entregable:** Vuelca los valores de los parámetros de amcl (comando `ros2 param dump /amcl`). Verás los que aparecen en el `amcl.yaml` pero también algunos otros. Investiga qué parámetros se relacionan con las cosas que hemos visto en clase de teoría sobre los filtros de bayes, modelos de movimiento o de sensor y filtros de partículas. Sobre el volcado de los parámetros explica brevemente su significado (solo de los relacionados con lo visto en teoría, por ejemplo no es el caso de `global_frame_id` que indica el nombre del sistema de coordenadas del mapa, por defecto `map`, eso es algo de ROS y no del filtro de partículas).

## Navegación (hasta 3 puntos)

Si a un mapa y un algoritmo de localización le añadimos una serie de algoritmos de planificación y seguimiento de trayectorias y evitación de obstáculos ya tendremos un sistema de navegación. Básicamente le podemos pedir al robot que llegue a un punto de manera autónoma.

Para que esto funcione, necesitarás tener un mapa del entorno y un archivo de configuración para la navegación, se suele llamar `nav2.yaml`. Verás que es mucho más largo y complejo que los de localización y mapeado (y por eso no se incluye el texto aquí).

Asegúrate que el AMCL que lanzaste en el paso anterior está parado, ya que la navegación lanza su propio AMCL. Para lanzar la navegación:

```
ros2 launch nav2_bringup bringup_launch.py map:=./mimapa.yaml params_file:=./nav2.y
```

Como siempre, si el mapa y/o el `nav2.yaml` no están en el directorio actual, cambia el `./` por la trayectoria para llegar hasta ellos.

En rviz tendrás que añadir el display del mapa igual que hiciste en el apartado de localización, clicar en la posición inicial del robot para que AMCL empiece a funcionar y una vez esté el robot

localizado puedes marcar la posición destino con el botón de la barra de herramientas `2D goal pose`.

Si todo va bien, tras un tiempo inicial en que el robot calcula la ruta, comenzará a moverse hacia su destino. Si quieres ver la ruta que ha calculado, añade un display `by topic` del topic `/plan` y de tipo `Path`.

Ten en cuenta que si lo intentas llevar por sitios demasiado estrechos o le das un destino demasiado cerca de un obstáculo es posible que el robot no encuentre un camino ya que siempre actúa con un cierto margen de seguridad (configurable en el `nav2.yaml`)

**Entregable:** incluye un video en el que se vea cómo marcas una posición destino y el robot va de forma autónoma hacia ella. Asegúrate de que se ve tanto la ventana del simulador como la de rviz.

**Entregable:** como puedes ver si examinas el `nav2.yaml` navegar es una tarea compleja que requiere de múltiples componentes en ROS. Aunque no es totalmente exacto podríamos decir que cada elemento del "nivel superior" del YAML (lo que está más a la izquierda) es como un componente de nav2. Así tendríamos `amcl`, `local costmap`, `global costmap`, `planner server`,... **Escoge dos de ellos** (salvo `amcl` que es del apartado anterior) e investiga qué hacen en líneas generales, qué significan los parámetros, y qué podrías cambiar en los valores si cambiara el entorno donde se mueve el robot o quisieras cambiar su comportamiento. Incluye toda esta información en la documentación

## Parte adicional: probar el mapeado y navegación en los robots reales (hasta 1 punto)

**La semana del 27 de octubre al 2 de noviembre y del 3 al 10 de noviembre** se probarán el mapeado, la localización y la navegación en los Turtlebot reales. Se harán turnos aleatorios para que se puedan usar los robots en grupos de 3-4 personas, **os puede tocar hacer la prueba uno de estos dos días, al azar**. La lista de turnos aparecerá publicada durante la semana inicial de la práctica. Si una vez asignado el turno no podéis venir por algún motivo justificado, consultadlo con el profesor.

Se publicará una guía indicando los pasos exactos para probar mapeado y navegación en los Turtlebot. En esta guía se indicará qué pruebas debéis hacer y cómo documentarlas. ~~La entrega de estas pruebas se realizará junto con la memoria principal de la práctica~~. Todos los que hayáis probado juntos el robot tendréis la misma puntuación.

## Parte adicional: análisis de los modelos de movimiento y de observación de ROS (hasta 1 punto)

Como hemos visto en clase de teoría, para poder implementar localización y mapeado basado en filtros de Bayes, que es lo que usa ROS por defecto, necesitamos primero un modelo

probabilístico de movimiento y del sensor.

Como el código fuente de ROS está disponible en Internet, en este apartado se trata de que lo examines y compares con lo visto en clase de teoría. Explica lo que hace el código de la forma más detallada que puedas y referencia en todo lo posible las transparencias de teoría relevantes (por ejemplo, “estas líneas calculan la probabilidad de obstáculos inesperados, como se hace en la figura de la derecha de la transparencia X del tema Y…”).

- El modelo de observación o del sensor basado en “haz de luz” o beam está en https://api.nav2.org/nav2-humble/html/beam__model_8cpp_source.html.
- El modelo de movimiento muestreado está en https://api.nav2.org/nav2-humble/html/differential__motion__model_8cpp_source.html .

Para la máxima nota en este apartado incluye información no solo del *beam* model o basado en haz de luz como lo llamábamos en clase de teoría sino también del *likelihood field model*, que también vimos en teoría.

> Para resolver esta parte os recomendamos que hagáis uso de ChatGPT o algún otro LLM (Modelo del Lenguaje) como Claude o Gemini. Copiad y pegad el código relevante y pedid que os lo explique, lo suelen hacer bastante bien. Como siempre que uséis un modelo del lenguaje, chequead con lo visto en teoría o con otras fuentes que la explicación tiene sentido y no está inventando cosas.

## Parte adicional: análisis de un filtro de partículas para localización (hasta 1 punto)

Probad alguna implementación ya hecha de un algoritmo de filtro de partículas para localización. No es necesario que esté integrada en ROS y puede estar implementada en cualquier lenguaje. En Internet hay multitud de ellas. Busca alguna para la que esté disponible el código, y conozcas el lenguaje de programación, ya que deberás examinar su funcionamiento.

Deberías

- Realizar varias pruebas con el algoritmo variando los parámetros que puedas (como mínimo el número de partículas)
- Explicar con el máximo detalle que puedas cómo funciona el código identificando las partes que hemos visro en clase (modelo del sensor, modelo de movimiento, bucle del algoritmo, resampling…). Hazlo en forma de comentarios al código fuente.
- Graba un vídeo de máximo 5 minutos explicando el código y haciendo pruebas del algoritmo. No es necesario que salgas tú directamente :) solo la pantalla

## Entrega de la práctica

### Baremo

En resumen, el baremo de puntuación es el siguiente:

- **Mapeado, localización y navegación (hasta un 7)**: la práctica debe estar adecuadamente documentada, detallando los resultados de todos los experimentos realizados.
- **(hasta 1 punto adicional)** probar el mapeado en los Turtlebot reales.
- **(hasta 1 punto adicional)** Análisis de los modelos de movimiento y de observación en el código fuente de ROS
- **(hasta 1 punto adicional)**: probar alguna implementación ya hecha de un algoritmo de filtro de partículas para localización.

En lugar de alguno de los puntos anteriores se puede hacer cualquier otra ampliación que se os ocurra, por ejemplo se podría valorar con hasta 1 punto probar algún otro algoritmo de mapeado o localización que esté implementado en ROS y esté disponible en Internet. En la memoria de la práctica deberíais incluir cómo lo habéis instalado, una breve explicación de las ideas en que se basa y resultados de funcionamiento, comparándolo con el algoritmo por defecto de ROS e indicando también casos en los que pueda fallar si así ha sido. **Consultad con el profesor** otras ideas alternativas para ver cuánto se podría valorar en el baremo

## Plazos y procedimiento de entrega

La práctica se podrá entregar hasta las 23:59 horas del **domingo 9 de Noviembre**.